

Macros — A Readable Companion to the Elixir Docs

Chapter 9 · checked against Elixir 1.20 · examples run on Elixir 1.14 · Sep 26, 2026

1. Why macros exist, and why to avoid them

A macro is a function that runs at compile time, receives code as data (an AST), and returns new code to put in its place. Much of Elixir itself is macros: `if`, `def`, `>`, `use`, and the test macros in `ExUnit`. They let libraries like `Ecto` and `Phoenix` offer syntax that reads like a language of its own.

The docs are blunt about using them yourself: “Macros should only be used as a last resort,” because “explicit is better than implicit” and “clear code is better than concise code.” A macro makes code harder to read, to debug and to compile quickly, and its mistakes don’t look like mistakes at the call site. Official anti-pattern pages cover five macro-specific problems. Assistants reach for macros readily, because they are impressive, so this chapter is about recognising when one isn’t needed and getting it right when it is.

2. The building blocks

Tool	Meaning
<code>quote do ... end</code>	Turn code into its AST without running it
<code>unquote(x)</code>	Insert a value or AST into a quote
<code>quote bind_quoted: [v: expr]</code>	Evaluate <code>expr</code> once, bind it to <code>v</code> inside the quote
<code>defmacro</code>	Define a macro; arguments arrive as AST
<code>require Mod</code>	Needed before calling <code>Mod</code> ’s macros
<code>use Mod, opts</code>	Calls <code>Mod.__using__(opts)</code> and inserts whatever code it returns
<code>var!(x)</code>	Deliberately break hygiene to reach a caller’s variable
<code>Macro.to_string/1</code> , <code>Macro.expand/2</code>	Inspect what a macro generates

Macro or function?

Use a function when	A macro may be justified when
The inputs are values	You need the <i>code</i> of an argument (to show it in an error, like <code>assert</code>)
Behaviour happens at run time	Work must happen at compile time (generating functions from a schema)

Use a function when	A macro may be justified when
You want to call it dynamically or pass it around	You are building a DSL whose syntax is the point

3. How it actually works

Arguments are code, not values

A macro receives the AST of its arguments. Every `unquote(arg)` in the result places that *code* in the output, so if the macro uses an argument twice, the expression runs twice. `bind_quoted` avoids this by evaluating each argument once, at the start of the generated code.

Hygiene

Variables created inside a `quote` belong to the macro's context, not the caller's. A macro that writes `x = 1` does not define `x` for the code around it. That is usually what you want; `var!/1` overrides it, and makes the macro's effect harder to see.

Compile-time dependencies

When a module uses a macro, the modules referenced in the macro's arguments can become *compile-time* dependencies: changing one forces recompiling the other. In a large project a few such macros can make every small edit recompile hundreds of files. `mix xref graph --label compile-connected` shows these chains.

use can inject anything

`use Mod` runs `Mod.__using__/1`, which can import functions, alias modules, define functions, set attributes and register callbacks, none of it visible at the call site. The anti-pattern guide recommends `import` and `alias` instead, where they are enough.

4. Plausible but wrong

4.1 A macro that should be a function

```
defmacro cents(amount) do
  quote do: round(unquote(amount) * 100)
end
```

There is nothing here a function can't do: no code inspection, no compile-time work. It can't be passed as `¢s/1`, needs `require`, and makes stack traces point at generated code. The anti-pattern guide calls this *unnecessary macros*. Fix: `def cents(amount), do: round(amount * 100)`.

4.2 Evaluating an argument twice

```
defmacro positive!(expr) do
  quote do
    if unquote(expr) > 0, do: unquote(expr), else: raise("not positive")
  end
end

positive!(next_id())
```

`next_id()` is called twice: once for the check, once for the result. With a counter it returned 2 and was called twice (verified). With a database insert or an HTTP call, the side effect happens twice. Fix: `quote bind_quoted: [value: expr]` and use `value`. That call returned 1, called once (verified).

4.3 Expecting a macro to set a caller's variable

```
defmacro load_config, do: quote(do: config = Application.get_all_env(:my_app))

def start do
  load_config()
  connect(config)
end
```

Hygiene keeps `config` inside the macro. The caller's `config` doesn't exist, and compilation fails with an undefined `config` (verified with a similar macro). The tempting fix, `var!(config)`, compiles but hides a variable binding inside a macro call. Fix: return a value (`config = load_config()`), and at that point a function will do.

4.4 Runtime values read at compile time

```
defmacro api_url do
  url = System.get_env("API_URL")
  quote do: unquote(url)
end
```

Code outside `quote` runs when the *caller* is compiled, so the URL from the build machine's environment is baked into the `.beam`, and production silently uses it. Fix: keep runtime lookups inside the generated code (or better, in a plain function), and read environment variables in `config/runtime.exs` (chapter 10).

4.5 use as a grab bag

```
defmodule MyApp.Web do
  defmacro __using__(_) do
    quote do
      import MyApp.Helpers
      import MyApp.Formatting
      alias MyApp.{Repo, Accounts, Billing}
      require Logger
    end
  end
end
```

```
end
end
end
```

Every module that says `use MyApp.Web` silently gets dozens of imports and aliases. Name clashes appear far from their cause, and readers can't tell where a function comes from. The anti-pattern guide calls this *"use" instead of "import"*. Fix: explicit `import / alias` in each module; keep `use` for cases where code must genuinely be injected, and document what it injects.

4.6 Large generated code

```
defmacro defroute(path, handler) do
  quote do
    def handle(unquote(path), conn) do
      # 60 lines of validation, parsing and error handling
    end
  end
end
```

Every call copies all 60 lines into the caller. A few hundred routes means tens of thousands of generated lines, slow compiles and large beams. The anti-pattern guide calls this *large code generation*. Fix: generate only the dispatch clause, and call an ordinary function for the work: `def handle(unquote(path), conn), do: Router.run(unquote(handler), conn)`.

4.7 Module names built at compile time from strings

```
for name <- ~w(user order invoice) do
  mod = Module.concat([MyApp.Schemas, Macro.camelize(name)])
  def schema(unquote(name)), do: unquote(mod)
end
```

The compiler can't see that this module depends on `MyApp.Schemas.User` and the others, so changes don't trigger the right recompilation, and missing modules surface only at runtime. The anti-pattern guide calls this *untracked compile-time dependencies*. Fix: write the module names out (`def schema("user"), do: MyApp.Schemas.User`) or keep an explicit map.

5. Review checklist and sources

- ☐ Every macro needs compile-time behaviour or the code of its arguments; otherwise it is a function
- ☐ Macros bind their arguments once (`bind_quoted`, or `unquote` each argument exactly once)
- ☐ No macro relies on setting the caller's variables; `var!` is justified where used
- ☐ No environment or configuration is read outside `quote` in a macro
- ☐ `use` is reserved for real code injection and documents what it injects; otherwise `import/alias`
- ☐ Generated code is small and delegates to functions
- ☐ Module names are written explicitly; compile-time dependency chains are checked with `mix xref`

Sources

- Macros: <https://hexdocs.pm/elixir/macros.html>
- Quote and unquote: <https://hexdocs.pm/elixir/quote-and-unquote.html>
- Meta-programming anti-patterns: <https://hexdocs.pm/elixir/macro-anti-patterns.html>
- Kernel.SpecialForms (quote): <https://hexdocs.pm/elixir/Kernel.SpecialForms.html#quote/2>